

Module 2: Transport Layer and Network Layer



Module 2: Transport Layer and Network Layer

Transport Layer

- Services and Protocols
- UDP and TCP

Hands-on

- Sockets Introduction
- Elementary TCP Sockets
- TCP Client/Server Example
- I/O Multiplexing
 - select()
 - poll()
- Elementary UDP Sockets
- Advanced I/O Functions

Network Layer

- Introduction
- Network-layer Protocols
- Unicast Routing
- Multicast Routing
- Multicasting Basics
- Intra-domain Routing
- Inter-domain Routing
- Next Generation IP (IPv6)
- Quality of Service (QoS)

Hands-on

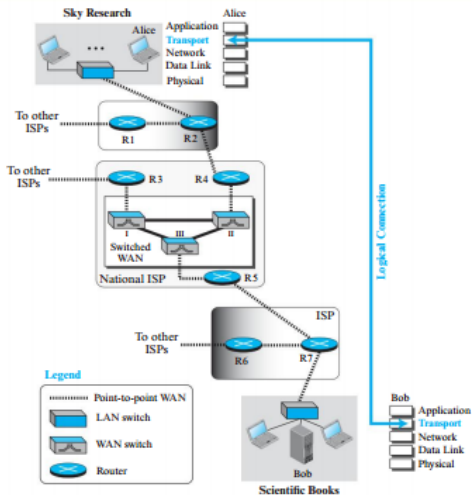
- Linux Routing Table
- Routing Caches
- Routing Cache Implementation
- Adding Routes using ip

Introduction to the Transport Layer

- The **Transport Layer** is located between:
 - Application Layer
 - Network Layer
- It provides **process-to-process communication** between:
 - A local application process
 - A remote application process
- Communication is achieved using a **logical connection**
- A logical connection means:
 - Applications assume a direct connection exists
 - Messages can be sent and received as if directly connected

Logical connection at the transport layer

Figure 3.1 Logical connection at the transport layer



Logical Connection in the Transport Layer

Alice's Host $\longleftrightarrow$ Bob's Host

- Example:
 - Alice at Sky Research Company
 - Bob at Scientific Books Company
- Both hosts communicate as though a direct connection exists between them
- In reality:
 - Data passes through multiple routers and networks
 - Routers use only the first three layers:
 - Physical Layer
 - Data Link Layer
 - Network Layer
- Only the **end systems** use the Transport Layer services

Transport-Layer Services

- The **Transport Layer** is located between:
 - Application Layer
 - Network Layer
- Responsibilities:
 - Provides services to the Application Layer
 - Receives services from the Network Layer
- Major Services Provided by the Transport Layer:
 - 1 Process-to-Process Communication
 - 2 Addressing using Port Numbers
 - 3 Encapsulation and Decapsulation
 - 4 Multiplexing and Demultiplexing
 - 5 Flow Control
 - 6 Error Control
 - 7 Congestion Control
 - 8 Connectionless Services
 - 9 Connection-Oriented Services

Process-to-Process Communication

- The first duty of the **Transport Layer** is to provide:
 - **Process-to-Process Communication**
- A **process** is:
 - An application-layer entity
 - A running program using transport-layer services
- Examples of processes:
 - Web browser
 - Email application
 - File transfer application

Host-to-Host vs Process-to-Process Communication(Process-to-Process Communication)

Network Layer

- Provides:
 - Host-to-Host Communication
- Delivers data:
 - Only to the destination computer
- Communication at:
 - Computer level

Transport Layer

- Provides:
 - Process-to-Process Communication
- Delivers data:
 - To the correct process
- Communication at:
 - Application level

Important Point

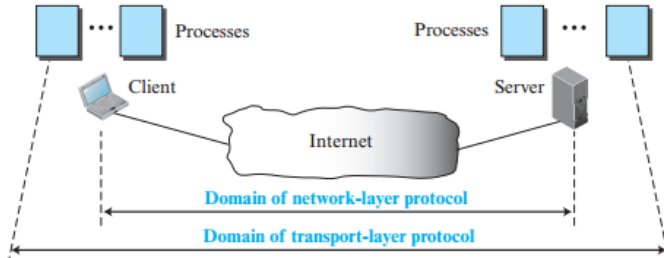
Delivery to the destination host alone is incomplete.
The message must also reach the correct application process.

Role of the Transport Layer(Process-to-Process Communication)

- The Network Layer identifies:
 - The destination computer (host)
- The Transport Layer identifies:
 - The destination process or application
- Therefore:
 - Network Layer → Host Delivery
 - Transport Layer → Process Delivery
- Example:
 - A computer may run:
 - Web server
 - Email server
 - FTP server
 - Transport Layer ensures data reaches the correct application

Network layer versus transport layer(Process-to-Process Communication)

Figure 3.2 *Network layer versus transport layer*

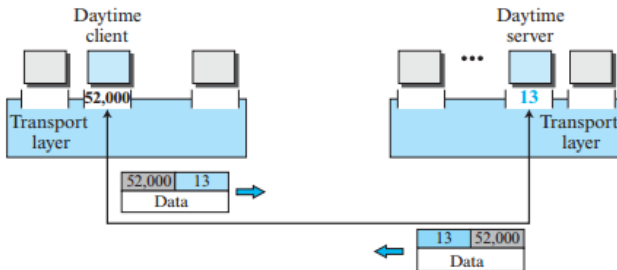


Addressing: Port Numbers — Client-Server Communication

- Process-to-process communication commonly uses the:
 - **Client-Server Paradigm**
- **Client**
 - Requests services
 - Usually runs on the local host
- **Server**
 - Provides services
 - Usually runs on the remote host
- Example:
 - Daytime Client requests date and time
 - Daytime Server sends date and time

Port Numbers

Figure 3.3 *Port numbers*



Addressing: Port Numbers — Need for Port Numbers

- Modern operating systems support:
 - Multiuser environments
 - Multiprogramming environments
- A computer can run:
 - Multiple server programs
 - Multiple client programs
- Communication requires identification of:
 - Local host
 - Remote host
 - Local process
 - Remote process
- Hosts are identified using:
 - IP Addresses
- Processes are identified using:
 - **Port Numbers**

Addressing: Port Numbers — Port Number Range

- In TCP/IP, port numbers are:
 - 16-bit integers
 - Range: 0 to 65,535
- Port numbers uniquely identify:
 - Application processes on a host

Important Idea

IP Address identifies the host

Port Number identifies the process

Addressing: Port Numbers — Ephemeral Port Numbers

- Client programs use:
 - **Ephemeral Port Numbers**
- Meaning of Ephemeral:
 - Short-lived or temporary
- Characteristics:
 - Assigned temporarily
 - Usually greater than 1023
 - Used during client communication
- Example:
 - Client Port Number = 52000

Addressing: Port Numbers — Well-Known Port Numbers

- Server processes use:
 - **Well-Known Port Numbers**
- Why not random numbers?
 - Clients must know the server port number
 - Random numbers create extra overhead
- Well-known ports are:
 - Permanent
 - Universally defined
- Example:
 - Daytime Server uses Port 13

Addressing: Port Numbers — IP Address vs Port Number

IP Address	Port Number
Identifies Host	Identifies Process
Host Selection	Process Selection
Used by Network Layer	Used by Transport Layer

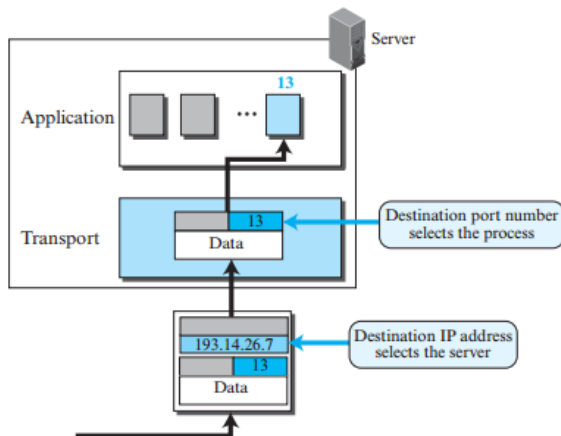
Example

192.168.1.10 : 80

- 192.168.1.10 → Destination Host
- Port 80 → Web Server Process

IP addresses versus port numbers

Figure 3.4 *IP addresses versus port numbers*



Addressing: Port Numbers — ICANN Port Ranges

- ICANN:
 - Internet Corporation for Assigned Names and Numbers
- ICANN divides port numbers into three categories:
 - 1 Well-Known Ports
 - 2 Registered Ports
 - 3 Dynamic (Private) Ports



Port Range	Category
0 – 1023	Well-Known Ports
1024 – 49151	Registered Ports
49152 – 65535	Dynamic/Private Ports

Addressing: Port Numbers — Well-Known Ports

- Range:
 - 0 – 1023
- Characteristics:
 - Assigned and controlled by ICANN
 - Used by standard server applications
- Examples:
 - HTTP → Port 80
 - HTTPS → Port 443
 - FTP → Port 21
 - DNS → Port 53

Addressing: Port Numbers — Registered Ports

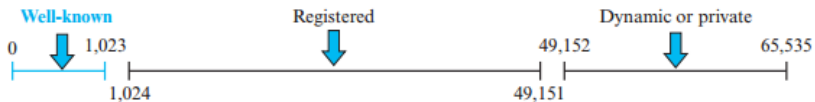
- Range:
 - 1024 – 49151
- Characteristics:
 - Not assigned directly by ICANN
 - Can be registered to avoid duplication
- Commonly used by:
 - User applications
 - Vendor-specific services

Addressing: Port Numbers — Dynamic/Private Ports

- Range:
 - 49152 – 65535
- Characteristics:
 - Neither controlled nor registered
 - Used temporarily by clients
- Also called:
 - Ephemeral Ports
 - Private Ports
- Mainly used for:
 - Temporary client communication

ICANN ranges

Figure 3.5 *ICANN ranges*



Addressing: Port Numbers — Example 3.1

- In UNIX systems, well-known ports are stored in:

File

/etc/services

- Each line in the file contains:
 - Server/Application name
 - Well-known port number
 - Protocol type (TCP/UDP)
- The `grep` utility is used to search entries

Addressing: Port Numbers — TFTP Port Example

Command

```
$ grep tftp /etc/services
```

Output

```
tftp    69/tcp  
tftp    69/udp
```

- TFTP:
 - Trivial File Transfer Protocol
- Uses:
 - Port 69
- Can operate using:
 - TCP
 - UDP

Addressing: Port Numbers — SNMP Port Example

Command

```
$ grep snmp /etc/services
```

Output

```
snmp      161/tcp  # Simple Net Mgmt Proto
snmp      161/udp  # Simple Net Mgmt Proto
snmptrap  162/udp  # Traps for SNMP
```

- SNMP:
 - Simple Network Management Protocol
- Uses two port numbers:
 - Port 161 → Normal SNMP communication
 - Port 162 → SNMP trap messages

Addressing: Port Numbers — Socket Addresses

- A transport-layer connection requires:
 - IP Address
 - Port Number
- The combination of:

Socket Address

IP Address + Port Number

- A socket address uniquely identifies:
 - A process on a host

Addressing: Port Numbers — Client and Server Socket Addresses

- Two socket addresses are needed for communication:
 - 1 Client Socket Address
 - 2 Server Socket Address
- Client Socket Address:
 - Identifies the client process uniquely
- Server Socket Address:
 - Identifies the server process uniquely

Client $\longleftrightarrow$ **Server**
(IP + Port) (IP + Port)

Addressing: Port Numbers — Components of Communication

- Internet communication requires four pieces of information:

- ① Source IP Address
- ② Destination IP Address
- ③ Source Port Number
- ④ Destination Port Number

- These values are stored in:

- Network-layer header → IP Addresses
- Transport-layer header → Port Numbers

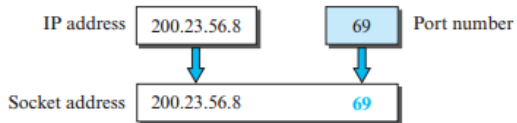
Example

192.168.1.10 : 52000 → Client Socket Address

172.16.1.5 : 80 → Server Socket Address

Socket Address

Figure 3.6 *Socket address*



Encapsulation and Decapsulation — Introduction

- To send data between processes, the Transport Layer performs:
 - ① Encapsulation
 - ② Decapsulation
- Encapsulation:
 - Happens at the sender side
- Decapsulation:
 - Happens at the receiver side

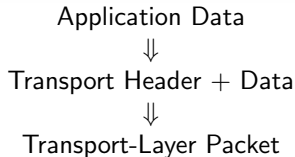


Encapsulation and Decapsulation — Encapsulation Process

- At the sender side:
 - ① Application process creates a message
 - ② Message is passed to the Transport Layer
 - ③ Socket addresses and protocol information are added
 - ④ Transport Layer adds a Transport Header
- This process is called:

Encapsulation

Adding protocol headers to the data before transmission



Encapsulation and Decapsulation — Transport-Layer Packets

- Transport-layer packets have different names depending on the protocol:

Protocol	Packet Name
UDP	User Datagram
TCP	Segment
General Term	Packet

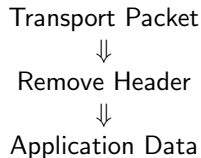
- These packets are forwarded to the Network Layer for transmission

Encapsulation and Decapsulation — Decapsulation Process

- At the receiver side:
 - ① Packet reaches the Transport Layer
 - ② Transport Header is removed
 - ③ Original message is extracted
 - ④ Message is delivered to the correct process
- This process is called:

Decapsulation

Removing protocol headers to recover the original data

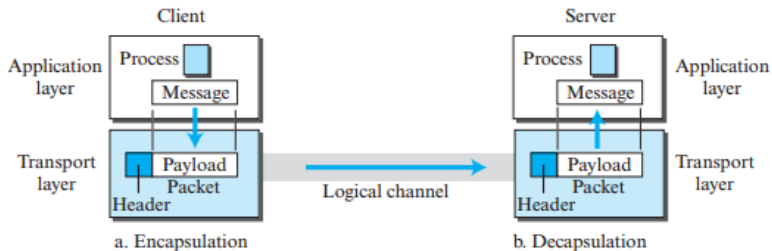


Encapsulation and Decapsulation — Important Points

- The sender socket address is also delivered to the receiver process
- This allows the receiver:
 - To identify the sender
 - To send a reply if needed
- Encapsulation and decapsulation ensure:
 - Proper communication between applications
 - Reliable transfer of messages
 - Correct process delivery

Encapsulation and decapsulation

Figure 3.7 *Encapsulation and decapsulation*



Multiplexing and Demultiplexing — Introduction

- **Multiplexing**

- Many-to-One process
- An entity accepts data from multiple sources

- **Demultiplexing**

- One-to-Many process
- An entity delivers data to multiple destinations

- In the Transport Layer:

- Source side performs Multiplexing
- Destination side performs Demultiplexing

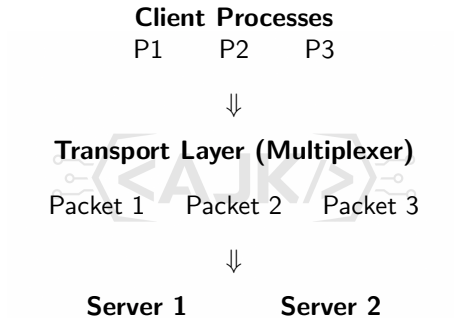
Multiplexing and Demultiplexing — Multiplexing at Source

- Multiple client processes can send data simultaneously
- Example:
 - Client processes:
 - P1
 - P2
 - P3
- The Transport Layer:
 - Accepts messages from all processes
 - Creates separate packets
 - Sends packets through the network
- Therefore, the Transport Layer acts as:

Multiplexer

Combines messages from many processes into packets

Multiplexing and Demultiplexing — Communication Example



- Packets 1 and 3 go to Server 1
- Packet 2 goes to Server 2

Multiplexing and Demultiplexing — Demultiplexing at Destination

- At the destination:
 - Transport Layer receives packets
 - Packets are delivered to correct processes
- The Transport Layer acts as:

Demultiplexer

Separates incoming packets and delivers them to appropriate processes

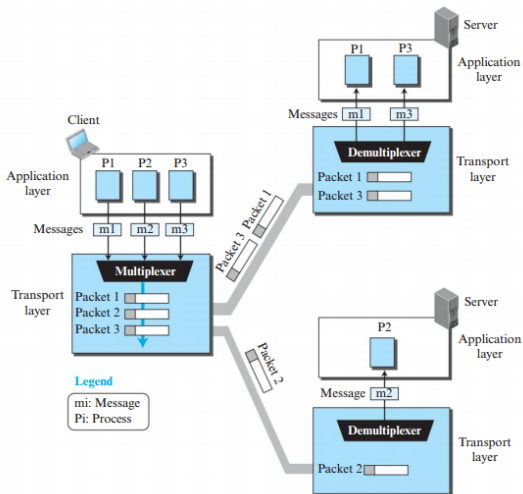
- Example:
 - Packet 1 → Process P1
 - Packet 3 → Process P3

Multiplexing and Demultiplexing — Important Points

- Multiplexing:
 - Many processes share the same communication channel
- Demultiplexing:
 - Incoming packets are delivered to correct applications
- Even if only one message arrives:
 - Demultiplexing is still required
- Port numbers help:
 - Identify the correct destination process

Multiplexing and Demultiplexing

Figure 3.8 Multiplexing and demultiplexing



Flow Control — Introduction

- Flow Control maintains balance between:
 - Data production rate
 - Data consumption rate
- Problem 1:
 - Producer sends data too fast
 - Consumer becomes overwhelmed
 - Some data may be lost
- Problem 2:
 - Producer sends data too slowly
 - Consumer must wait
 - System efficiency decreases

Goal of Flow Control

Prevent data loss and maintain efficient communication

Flow Control — Pushing and Pulling

Pushing

- Producer sends data immediately
- No request from consumer
- Consumer may become overloaded
- Requires Flow Control

Pulling

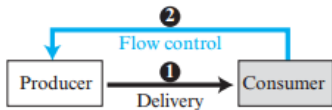
- Consumer requests data
- Producer sends only when requested
- Consumer receives data when ready
- No need for Flow Control

Key Point

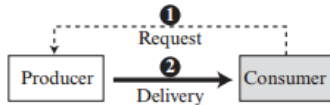
Flow control is mainly required in pushing systems.

Pushing and Pulling

Figure 3.9 *Pushing or pulling*



a. Pushing



b. Pulling

Flow Control — Communication at the Transport Layer

- Four entities participate in communication:

- 1 Sender Process
- 2 Sender Transport Layer
- 3 Receiver Transport Layer
- 4 Receiver Process

- Sender Process:

- Produces message chunks
- Pushes them to the Transport Layer

- Sender Transport Layer:

- Consumes messages
- Encapsulates them into packets
- Sends packets



Flow Control — Receiver Side Operations

- Receiver Transport Layer:

- Receives packets
- Decapsulates messages
- Delivers data to application layer

- Receiver Process:

- Pulls messages when ready

- Two flow-control situations exist:

- 1 Between Sending Application and Sending Transport Layer
- 2 Between Receiving Transport Layer and Sending Transport Layer

Flow Control — Buffers

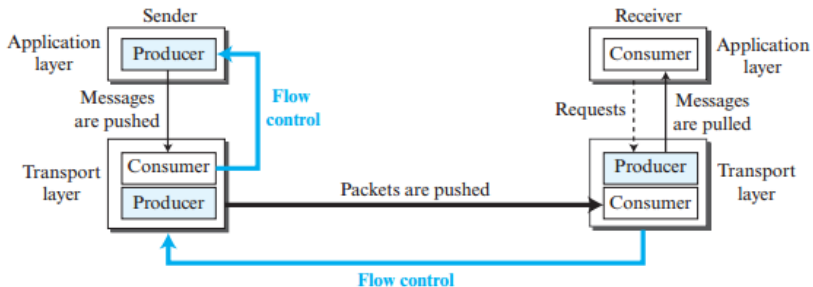
- Flow control commonly uses:
 - Buffers
- Buffer:
 - A memory area used to temporarily store packets
- Two buffers are generally used:
 - 1 Sender Transport Layer Buffer
 - 2 Receiver Transport Layer Buffer
- Buffers help:
 - Manage speed differences
 - Prevent packet loss

Flow Control — Buffer Communication

- If the sender buffer becomes full:
 - Transport Layer asks Application Layer to stop sending data
- When space becomes available:
 - Application Layer can resume sending data
- If the receiver buffer becomes full:
 - Receiver Transport Layer asks sender to stop transmission
- When vacancies are available:
 - Sender is informed to continue transmission

Flow Control At Transport Layer

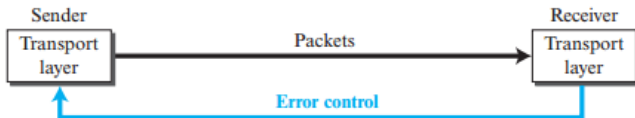
Figure 3.10 *Flow control at the transport layer*



Flow Control — Example 3.2

- Assume:
 - Each Transport Layer has only one buffer slot
- Sender Side:
 - Empty slot → Request next message chunk
- Receiver Side:
 - Empty slot → Send acknowledgment
- Problem:
 - Single-slot buffering is inefficient
 - Communication becomes slower

Figure 3.11 *Error control at the transport layer*



Error Control — Introduction

- The Internet Protocol (IP) at the Network Layer is:
 - Unreliable
- Therefore, the Transport Layer provides:

Error Control

Services that ensure reliable communication

- Error control is required when:
 - Applications need reliable data delivery

Error Control — Responsibilities

- Error control at the Transport Layer is responsible for:
 - 1 Detecting corrupted packets
 - 2 Discarding corrupted packets
 - 3 Detecting lost packets
 - 4 Resending lost packets
 - 5 Detecting duplicate packets
 - 6 Discarding duplicate packets
 - 7 Handling out-of-order packets
- Missing packets are:
 - Buffered until received

Error Control — Communication Between Transport Layers

- Error control operates between:
 - Sending Transport Layer
 - Receiving Transport Layer
- Unlike flow control:
 - Application Layer is not involved
- Assumption:
 - Communication between Application Layer and Transport Layer is error-free
- Receiver usually informs sender about:
 - Errors
 - Missing packets
 - Duplicate packets

Error Control — Sequence Numbers

- Error control uses:

Sequence Numbers

Numbers assigned to packets for tracking

- Sequence numbers help to:
 - Identify lost packets
 - Detect duplicate packets
 - Detect out-of-order packets
- Sequence number is stored in:
 - Transport-layer header

Error Control — Sequence Number Range

- Sequence numbers are limited by:
 - Number of bits available in the header

- If sequence number field has:

m bits

- Then sequence numbers range from:



- Example:

$$m = 4$$

- Sequence numbers:

$$0, 1, 2, \dots, 15$$

- Sequence numbers wrap around using:

Modulo Arithmetic

Sequence numbers are modulo 2^m

Error Control — Acknowledgment (ACK)

- Error control commonly uses:

ACK (Acknowledgment)

Positive signal sent by the receiver

- ACK indicates:
 - Packets arrived correctly
- Corrupted packets:
 - Are discarded by the receiver
- Sender uses:
 - Timers to detect packet loss

Error Control — Retransmission and Packet Handling

- Sender operation:
 - Starts timer after sending packet
- If ACK is not received before timeout:
 - Packet is retransmitted
- Duplicate packets:
 - Silently discarded by receiver
- Out-of-order packets:
 - May be discarded
 - Or buffered until missing packets arrive

Combination of Flow and Error Control — Introduction

- Flow control requires:
 - Buffers at sender and receiver
- Error control requires:
 - Sequence numbers
 - Acknowledgment numbers
- These mechanisms can be combined using:

Numbered Buffers

Buffers with sequence-numbered packet storage

Combination of Flow and Error Control — Sender Side Operation

- At the sender side:
 - ① Packet is prepared for transmission
 - ② Next free buffer location x is selected
 - ③ x becomes the sequence number
 - ④ Packet is sent
 - ⑤ Copy of packet is stored in buffer location x
- Stored packet waits for:
 - Acknowledgment (ACK)
- After ACK arrives:
 - Packet is removed from buffer
 - Memory location becomes free

Combination of Flow and Error Control — Receiver Side Operation

- At the receiver side:
 - ① Packet with sequence number y arrives
 - ② Packet is stored in buffer location y
 - ③ Packet waits until Application Layer is ready
- Receiver then:
 - Sends acknowledgment for packet y

Important Point

Buffer locations are identified using sequence numbers.

Combination of Flow and Error Control — Sliding Window Concept

- Sequence numbers use:

$$0 \text{ to } 2^m - 1$$

- Sequence numbers operate using:

- Modulo 2^m



- A sliding window:

- Represents a portion of the sequence-number space
 - Represents available buffer locations

- Window moves as packets are:

- Sent
 - Acknowledged

Combination of Flow and Error Control — Sliding Window at Sender

- When a packet is sent:
 - Corresponding window slice is marked
- If all slices are marked:
 - Sender buffer becomes full
 - No more packets can be accepted
- When ACK arrives:
 - Corresponding slice becomes unmarked
- Window slides forward:
 - New free buffer locations become available

Combination of Flow and Error Control — Sliding Window Example

- Example:

- $m = 4$
- Sequence numbers: 0 to 15
- Window size = 7



- Sliding window:

- Occupies part of the sequence-number circle
- Moves as communication progresses

- Sliding window is:

- A logical abstraction
- Implemented using variables in computers

Combination of Flow and Error Control — Circular and Linear Representation

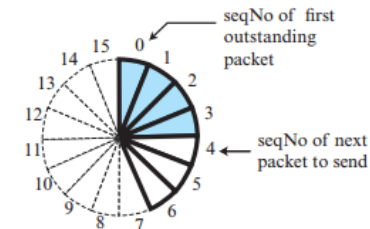
- Sliding window can be represented in two ways:
 - 1 Circular Representation
 - 2 Linear Representation
- Circular representation:
 - Uses a sequence-number circle
- Linear representation:
 - Easier to draw on paper
 - Commonly used in textbooks and protocols

Key Idea

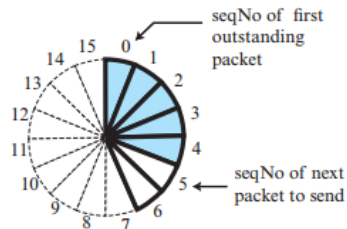
Both representations describe the same sliding-window mechanism.

Sliding window in circular format

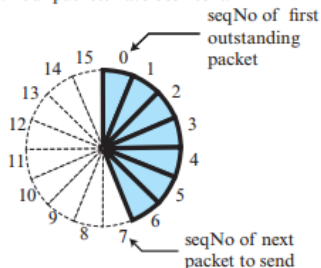
Figure 3.12 *Sliding window in circular format*



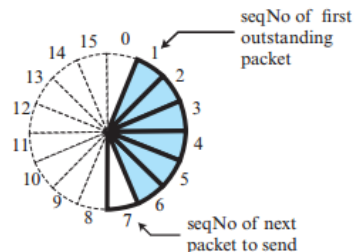
a. Four packets have been sent.



b. Five packets have been sent.



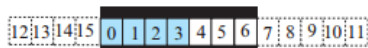
c. Seven packets have been sent;



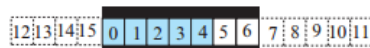
d. Packet 0 has been acknowledged;

Sliding window in linear format

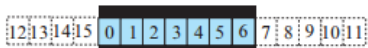
Figure 3.13 *Sliding window in linear format*



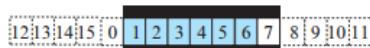
a. Four packets have been sent.



b. Five packets have been sent.



c. Seven packets have been sent;
window is full.



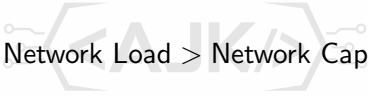
d. Packet 0 has been acknowledged;
window slides.

Congestion Control — Introduction

- Congestion is a major issue in:

- Packet-switched networks
- Example: Internet

- Congestion occurs when:


$$\text{Network Load} > \text{Network Capacity}$$

- Network Load:

- Number of packets entering the network

- Network Capacity:

- Number of packets the network can handle

Congestion Control — Definition

- **Congestion Control** refers to:

Definition

Mechanisms and techniques used to keep network load below network capacity

- Main goals:
 - Prevent network overload
 - Reduce packet loss
 - Improve network performance
 - Maintain smooth communication

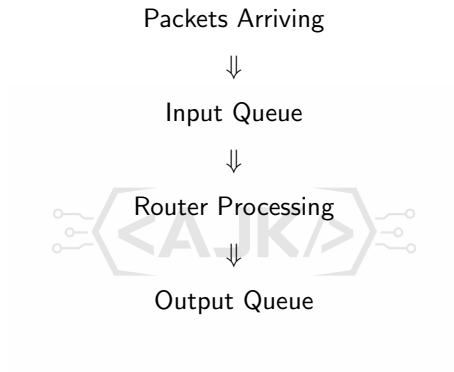
Congestion Control — Real-Life Analogy

- Congestion occurs in any system involving:
 - Waiting
 - Queues
- Example:
 - Traffic congestion on a freeway
- During rush hour:
 - Accident or blockage slows traffic
 - Vehicles accumulate
 - Traffic jam occurs
- Similar situation occurs in computer networks

Congestion Control — Causes of Congestion

- Routers and switches contain:
 - Queues (buffers)
- Queues temporarily store packets:
 - Before processing
 - After processing
- A router usually has:
 - Input Queue
 - Output Queue
- Congestion occurs when:
 - Packets arrive faster than they can be processed

Congestion Control — Queue Overload



- If processing is slower than arrival rate:
 - Queues become full
 - Packets are delayed or dropped
 - Congestion occurs

Congestion Control — Congestion at Different Layers

- Congestion primarily occurs at:
 - Network Layer
- Transport Layer experiences:
 - Effects of Network Layer congestion
- Example effects:
 - Packet loss
 - Delay
 - Retransmissions
- TCP implements:
 - Its own congestion control mechanisms
- This happens when:
 - Network Layer has no congestion control support



Connectionless and Connection-Oriented Services — Introduction

- Transport-layer protocols provide two types of services:
 - 1 Connectionless Service
 - 2 Connection-Oriented Service
- These services differ from Network-Layer services
- At the Transport Layer:
 - Connectionless → Independent packets
 - Connection-Oriented → Dependent packets

Important Point

Transport Layer assumes a logical connection between hosts.

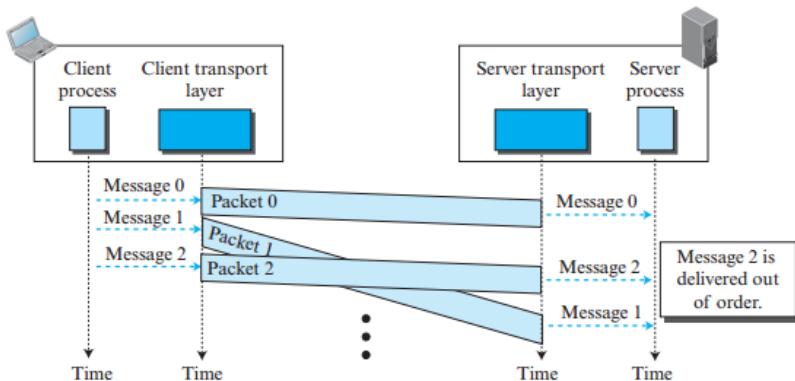
Connectionless and Connection-Oriented Services — Connectionless Service

- In a connectionless service:
 - Message is divided into chunks
 - Each chunk is treated independently
- Transport Layer:
 - Encapsulates each chunk separately
 - Sends packets independently
- No relationship exists between packets



Connectionless Service

Figure 3.14 *Connectionless service*



Connectionless and Connection-Oriented Services — Out-of-Order Delivery

- Example:

- Client sends packets:

0, 1, 2

- Due to network delays:

- Packets may arrive as:

0, 2, 1

- Result:

- Server receives data out of order
 - Message may become incorrect

Connectionless and Connection-Oriented Services — Problems in Connectionless Service

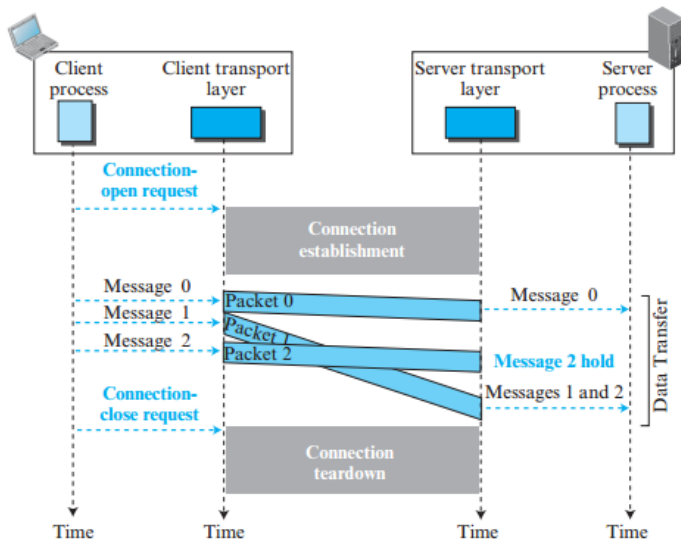
- If packets are lost:
 - Receiver cannot detect missing packets
- Reason:
 - Packets are not numbered
 - No coordination between transport layers
- Receiving Transport Layer does not know:
 - When packets start arriving
 - When all packets have arrived

Limitation

Effective flow control, error control, and congestion control are difficult to implement.

Connection Oriented Service

Figure 3.15 Connection-oriented service



Connectionless and Connection-Oriented Services — Connection-Oriented Service

- In a connection-oriented service:
 - Client and server establish a logical connection first
- Communication occurs in three phases:
 - 1 Connection Establishment
 - 2 Data Transfer
 - 3 Connection Termination (Tear Down)
- Data transfer begins only after:
 - Connection establishment

Connectionless and Connection-Oriented Services — End-to-End Connection

- At the Transport Layer:
 - Connection-oriented service is end-to-end
- Only two hosts participate:
 - Sender host
 - Receiver host
- Routers inside the network are not involved in:
 - Transport-layer connection management
- Transport-layer connection-oriented service can run over:
 - Connectionless Network Layer
 - Connection-Oriented Network Layer



Connectionless and Connection-Oriented Services — Advantages

- Connection-oriented protocols support:

- ① Flow Control
- ② Error Control
- ③ Congestion Control

- Benefits:

- Reliable delivery
- Ordered delivery
- Duplicate detection
- Better network management

Connectionless and Connection-Oriented Services — Finite State Machine (FSM)

- Transport-layer protocols can be modeled using:

Finite State Machine (FSM)

A machine with a finite number of states

- FSM characteristics:
 - Machine stays in one state
 - Events trigger actions
 - Events determine next state
- One state is defined as:
 - Initial State

Connectionless and Connection-Oriented Services — FSM in Connectionless Service

- Connectionless Transport Layer FSM:
 - Has only one state

Established State

Ready to send and receive packets at all times

- No connection setup or termination is required

Connectionless and Connection-Oriented Services — FSM in Connection-Oriented Service

- Connection-oriented FSM contains multiple states:

- Closed State
- Open-Wait-I
- Open-Wait-II
- Established State
- Close-Wait-I
- Close-Wait-II

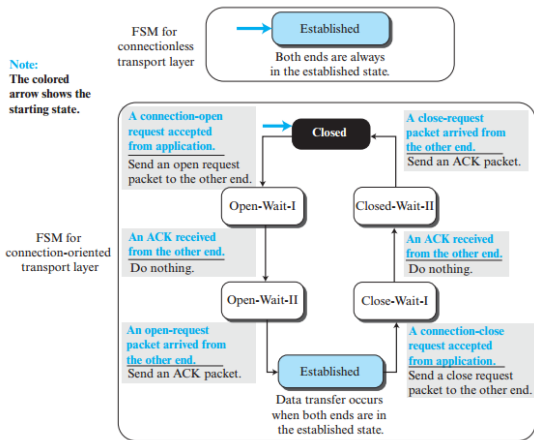


- States manage:

- Connection establishment
- Data transfer
- Connection termination

Connectionless and connection-oriented service represented as FSMs

Figure 3.16 Connectionless and connection-oriented service represented as FSMs



Connectionless and Connection-Oriented Services — Connection Establishment

- Initial state:
 - Closed State
- Connection establishment steps:
 - 1 Local process requests connection
 - 2 Open-request packet is sent
 - 3 ACK is received
 - 4 Connection becomes established
- Bidirectional communication begins in:

Established State

Both ends are ready for data exchange

Connectionless and Connection-Oriented Services — Connection Termination

- To close a connection:

- ① Application sends close request
- ② Transport Layer sends close-request packet
- ③ ACK is received
- ④ Close request from remote side is received
- ⑤ Final ACK is sent

- Final state:

Closed State

Connection is completely terminated

- FSM designs may vary:

- Different protocols use different FSM structures